

IMAGE SEGMENTATION



Curso 2025/2026

HECHO POR:

Nombre	NIA	Correo Electrónico	Grupo
María Montanet Estebaranz	100495891	100495891@alumnos.uc3m.es	84
Guillermo Sánchez Sanlucas	100495873	100495873@alumnos.uc3m.es	84
Lara Gómez Carrera	100495767	100495767@alumnos.uc3m.es	84

1. INTRODUCCIÓN

La evaluación automatizada de daños post-desastre mediante imágenes satelitales es crucial para la respuesta a emergencias. El conjunto de datos **xBD** se presenta como el estándar en este dominio, planteando un problema de segmentación semántica multiclase. Sin embargo, el principal obstáculo técnico de **xBD** es su desbalanceo de clases extremo: mientras que los edificios sin daños (no-damage) representan aproximadamente el 84% de las muestras, las categorías críticas como daños mayores (major-damage) y edificios destruidos (destroyed) apenas alcanzan el 5% y el 2%, respectivamente [1]. Un modelo de segmentación estándar, como *DeepLabV3*, optimizado mediante validación cruzada simple, tiende a colapsar rápidamente hacia la clase mayoritaria, ignorando las estructuras dañadas para minimizar la pérdida global.

Para establecer un punto de partida robusto (Baseline Compartido), implementamos una estrategia de mitigación a nivel de conjunto de datos. En primer lugar, se realizó una división estratificada (85/15) entre entrenamiento y validación, utilizando como pivote la clase de daño más grave presente en cada parche de imagen. En segundo lugar, se integró un *WeightedRandomSampler* en el *DataLoader* de entrenamiento, asignando pesos de muestreo inversamente proporcionales a la frecuencia de las clases.

2. Extensiones y Análisis Propuestos

2.1. Extensión de Datos: Aumento Espacial

El primer reto al que nos tuvimos que enfrentar, una vez habíamos desarrollado el baseline, fue el aumento espacial. El reto específico del aumento de datos en segmentación de imágenes es que cualquier transformación espacial, ya sea un recorte, un giro o una rotación, debe replicarse de forma idéntica sobre la imagen y su máscara de etiquetas simultáneamente. Aplicar la transformación solo a la imagen desalinea los píxeles respecto a sus etiquetas y corrompe los datos de entrenamiento. Es por esto que decidimos implementar la clase llamada *JointTransform* (recogida en el fichero “augment.py”) para aplicar las transformaciones espaciales (flip horizontal, flip vertical, rotación aleatoria de $\pm 15^\circ$ y *RandomResizedCrop*) con los mismos parámetros sobre la imagen post-desastre, la imagen pre-desastre y la máscara de forma sincronizada. Las transformaciones como la variación de brillo o contraste y el desenfoque gaussiano, en cambio, las aplicamos exclusivamente a las imágenes, ya que la máscara es una imagen de etiquetas discretas y alterar su apariencia visual no nos generaba ningún valor añadido al modelo pero tampoco ningún conflicto. Una vez aclarada la forma en la que íbamos a afrontar cada tipo de transformaciones tuvimos que, dado que operaciones como la rotación o el recorte requieren redimensionar los píxeles a nuevas posiciones, usar interpolación BILINEAR para las imágenes, que produce transiciones suaves, y NEAREST para la máscara, evitando así que la interpolación generase valores intermedios que no correspondieran a ninguna clase de daño real.

Con el objetivo de cuantificar la contribución individual de cada una de las técnicas, llevamos a cabo una ablación sobre cinco configuraciones (recogidas en la Figura [1] del anexo): *aug_none* (sin aumento, y el que tomábamos de referencia), *aug_flips* (flips en horizontal y vertical), *aug_geo* (flips más rotación), *aug_full* (flips, rotación, cambios de brillo y contraste, y desenfoque gaussiano) y *aug_crop* (*aug_full* más *RandomResizedCrop* al 50–100% del área, diseñada específicamente para simular distintas escalas de observación y mejorar la detección de clases minoritarias dispersas en el parche). Dentro de los resultados que obtuvimos, lo más llamativo fue que *aug_none* nos devolvía el mejor **Val mIoU**, siendo 0.6226. Este hallazgo nos resultó coherente dentro de los primeros experimentos completos que realizamos ya que con solo 10 épocas de entrenamiento y un scheduler *StepLR* que multiplica el *learning rate* por 0.1 en la época 5, los modelos con aumentos más agresivos no disponen del tiempo suficiente para que el efecto regularizador del aumento compense el incremento de dificultad en la convergencia. El modelo sin aumento sobreajustó más rápido y dominó en el corto plazo, lo cual nos indicó que la utilidad del aumento de datos estaba condicionada al presupuesto de entrenamiento disponible.

En experimentos posteriores comparamos la mejor configuración de aumento (*aug_crop*) con y sin el *WeightedRandomSampler* (*WRS*) para medir el efecto de combinar ambos mecanismos de mitigación del desbalanceo. Como se puede observar en la Tabla [1], la combinación produce una mejora sistemática en todas las clases minoritarias: +37% en **minor-damage**, +73% en **major-damage** y +12% en **destroyed**. Con esto pudimos confirmar que el aumento espacial y el *WRS* son mecanismos complementarios. El *WRS* actúa sobre qué parches ve el modelo durante el entrenamiento, mientras que el aumento actúa sobre cómo los ve. Sin embargo, a pesar de haber conseguido una mejora considerable dentro de esta parte del trabajo, para la solución final decidimos prescindir del *WRS* en este apartado ya que la función de pérdida empleada en el modelo final incorporaba un balanceo implícito de clases. En ese caso, aplicar ambos mecanismos simultáneamente nos supuso un doble balanceo que desestabilizó la convergencia del sistema en mayor medida de lo que el *WRS* beneficiaba en la sección de aumento de datos.

2.2. Extensión de Pérdida: Penalización de Clases Minoritarias

Para combatir el desbalanceo desde la función de pérdida, se diseñó el módulo “losses.py” con seis variantes (*Cross-Entropy*, *Weighted CE*, *Dice*, *Focal*, *Tversky*, *Focal-Tversky* y *Combo CE + Dice*), junto con un helper *compute_class_weights* con cuatro esquemas de ponderación. Nuestra hipótesis inicial era que sustituir *CE* plana

por una pérdida balanceada mejoraría la detección de las clases minoritarias por sí sola. Sin embargo, los primeros experimentos dieron un resultado inesperado: las pérdidas regionales colapsaban a $mIoU \approx 0,003$. Esto motivó una segunda fase de análisis sobre la interacción entre la función de pérdida y el muestreo del DataLoader.

Para estudiarlo, ejecutamos un bucle con 6 funciones de pérdida con 2 estrategias de muestreo, 10 configuraciones en total, entrenando cada combinación 8 épocas con $img_size=128$, $batch=4$, $AdamW + CosineAnnealing$ y semilla fija. Los resultados se visualizan en la Figura [2]. El hallazgo central es que el balanceo en sampler y el balanceo en pérdida son alternativas, no complementarias: combinarlos genera un sobremuestreo redundante que degrada el rendimiento. *Dice* y *Focal-Tversky* cayeron a $mIoU \approx 0,003$ con *WRS* activo (la red predice solo la clase y la *val_loss* permanece plana durante todas las épocas), mientras que sin *WRS Dice* se recuperó hasta $mIoU \approx 0,092$.

La única pérdida robusta a ambos regímenes resultó ser *Combo* ($\alpha \cdot CE + (1-\alpha) \cdot Dice$) con $\alpha=0.5$: la componente CE estabiliza el aprendizaje píxel a píxel evitando colapsos, mientras que la componente *Dice* optimiza directamente *IoU*, la métrica de evaluación. *Combo* obtuvo 0.1418 con *WRS* y 0.1453 sin *WRS*, una variación despreciable que confirma su robustez. El panel derecho de la Figura [2] muestra que el desempeño por clase de *Combo* (sin *WRS*) es muy similar al de *Cross-Entropy* plana con *WRS*, ambas dominadas por la clase mayoritaria **no-damage** y con valores casi nulos en daño grave. La razón para preferir *Combo* no es por tanto su superioridad numérica frente a CE, sino su robustez metodológica: es la única pérdida que mantiene su rendimiento al variar la muestra, lo que la hace una elección segura para el run final integrado donde otras decisiones del pipeline pueden interactuar con la pérdida de formas no anticipadas.

Al entrenar el modelo con la función de pérdida elegida, entrenado 15 épocas y evaluada sobre *test_with_labels* (5 desastres no vistos en entrenamiento), la matriz resultante (Figura [3]) revela que el modelo identifica **no-damage** con 70% de *recall* pero confunde sistemáticamente las clases de daño con **background** y **no-damage**: el 72% de píxeles **destroyed** se clasifican como **background**, y el 63% de **major-damage** como **no-damage**.

2.3. Extensión de Arquitectura: Optimización y Representación

Para elegir el mejor optimizador gastando poca memoria, hicimos pruebas rápidas usando una red más ligera (ResNet-18). Vimos que con SGD el modelo no aprendía nada (*mIoU* de 0.0) porque colapsaba y predecía siempre la clase mayoritaria (no-damage). En cambio, la combinación **AdamW + PolynomialLR** sí logró detectar daños rápidamente gracias a que adapta la tasa de aprendizaje de forma automática. Por ello, fijamos esta combinación como el "motor" definitivo para lanzar los experimentos principales de arquitectura usando ya la red ResNet-101.

Con el optimizador ya ajustado, dimos el salto a la red principal (ResNet-101) y pusimos a prueba cuatro versiones: el Baseline (sin modificar), un modelo con un clasificador auxiliar, uno congelando sus primeras capas y otro reduciendo las ventanas de visión (tasas ASPP) para captar detalles más pequeños.

Al evaluarlos sobre el conjunto de Test, observamos que modificar únicamente la arquitectura de la red no es suficiente para contrarrestar un desbalanceo de clases tan extremo. Aunque todas las versiones tuvieron dificultades para clasificar los niveles de daño más escasos, los modelos adaptados (en particular, la variante con las tasas ASPP reducidas) mostraron algo más de estabilidad al identificar los edificios destruidos en comparación con el modelo base. Esto demuestra que ajustar la estructura interna de la red ayuda a que el modelo capte mejor los detalles locales, aunque elevar las métricas globales siga siendo un desafío considerable.

3. Conclusiones y Solución Final

El proyecto ha explorado tres vías para combatir el desbalanceo extremo del dataset xBD: aumento de datos, función de pérdida y arquitectura. El hallazgo más relevante es que estas estrategias interactúan de formas no obvias. Con pocas épocas, el aumento agresivo perjudica la convergencia en lugar de mejorarla —*aug_none* superó a todas las configuraciones con sólo 10 épocas—, y combinar *WeightedRandomSampler* con pérdidas regionales como *Dice* o *Focal-Tversky* produce un doble balanceo que colapsa el modelo ($mIoU \approx 0,003$). La única pérdida robusta a ambos regímenes fue *Combo* ($CE + Dice$, $\alpha=0,5$), que mantuvo resultados estables con y sin *WRS* (0,1453 vs 0,1418), y se eligió como pérdida final precisamente por esa fiabilidad.

El modelo final, entrenado 15 épocas con *Combo* sin *WRS*, alcanza un *mIoU* de 0,1249 sobre el conjunto de test. El desglose refleja el límite del sistema: **no-damage** obtiene un *IoU* del 48,41% con un *recall* del 72,83%, pero las clases críticas son casi invisibles para el modelo —**major-damage** roza el 0,01% y **destroyed** el 0,54%—, con el 43% de sus píxeles clasificados como **background** y el 62,58% de **major-damage** confundido con **no-damage**. Las modificaciones arquitectónicas (tasas ASPP reducidas, clasificador auxiliar) y el optimizador **AdamW + PolynomialLR** aportaron estabilidad, pero no fueron suficientes para revertir este sesgo por sí solos.

En conjunto, los experimentos confirman que el preprocesamiento —especialmente el muestreo— tiene mayor

impacto que la arquitectura o la pérdida de forma aislada, y que cualquier mejora real en las clases minoritarias requiere combinar estas estrategias de forma cuidadosa, con un presupuesto de entrenamiento adecuado que permita amortizar el coste de regularización.

5. ANEXOS

TABLAS

Tabla 1:

Clase	Sin WRS	Con WRS	Mejora (%)
no-damage	0.4559	0.4438	-2.7%
minor-damage	0.0554	0.0760	+37.3%
major-damage	0.0194	0.0335	+72.7%
destroyed	0.0299	0.0336	+12.4%
mIoU (4 clases)	0.1401	0.1467	+4.7%

Efecto del WeightedRandomSampler sobre el IoU por clase al combinar aug_crop con y sin muestreo ponderado (10 épocas). La incorporación del WRS mejora sistemáticamente todas las clases minoritarias (+37.3% en minor-damage, +72.7% en major-damage, +12.4% en destroyed) a costa de una ligera reducción en no-damage (-2.7%), lo que evidencia que el WRS redistribuye efectivamente la atención del modelo desde la clase dominante hacia las clases críticas.

FIGURAS

Figura 1:

Resultados de la ablación de Data Augmentation con 5 configuraciones (10 épocas c/u). Panel izquierdo: Val mIoU por configuración de aumento, donde se observa que aug_none obtiene el mejor resultado (0.6226) y el rendimiento decrece progresivamente al añadir transformaciones más agresivas. Panel derecho: curvas de entrenamiento por configuración, donde el pico visible en la época 6 refleja la caída del learning rate impuesta por el StepLR ($\gamma=0.1$), que limita el horizonte de aprendizaje de las augmentaciones más complejas.

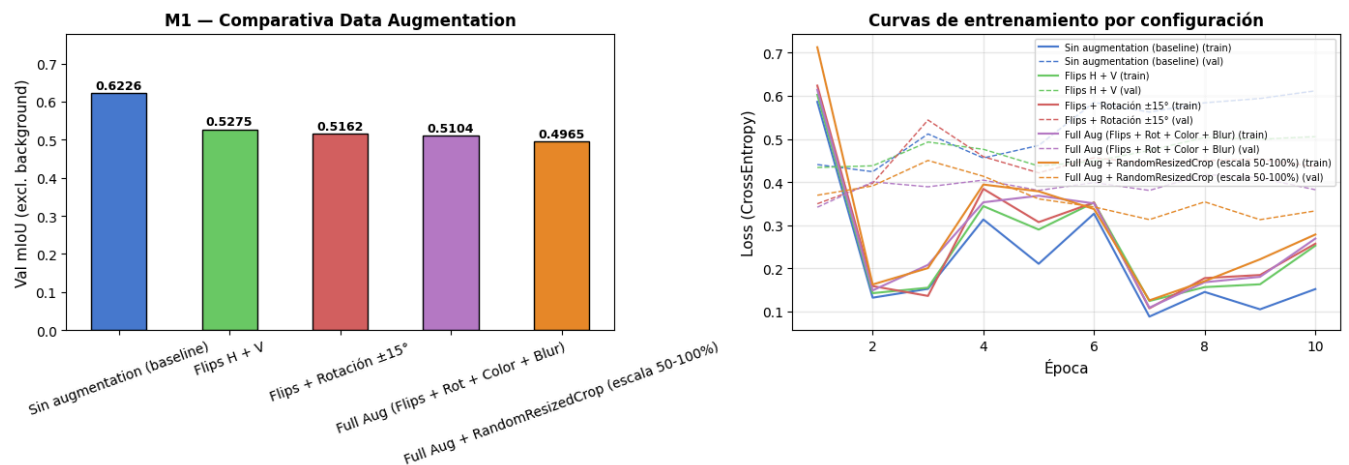


Figura 2:

Resultados de la ablación 2D loss $\times$ sampler. Panel izquierdo: mIoU foreground por función de pérdida con cada estrategia de muestreo (con WRS vs. shuffle), donde se observa el colapso de las pérdidas regionales con WRS activo y la robustez de Combo. Panel derecho: desglose por clase del IoU obtenido por las dos mejores configuraciones (CE con WRS y Combo sin WRS).

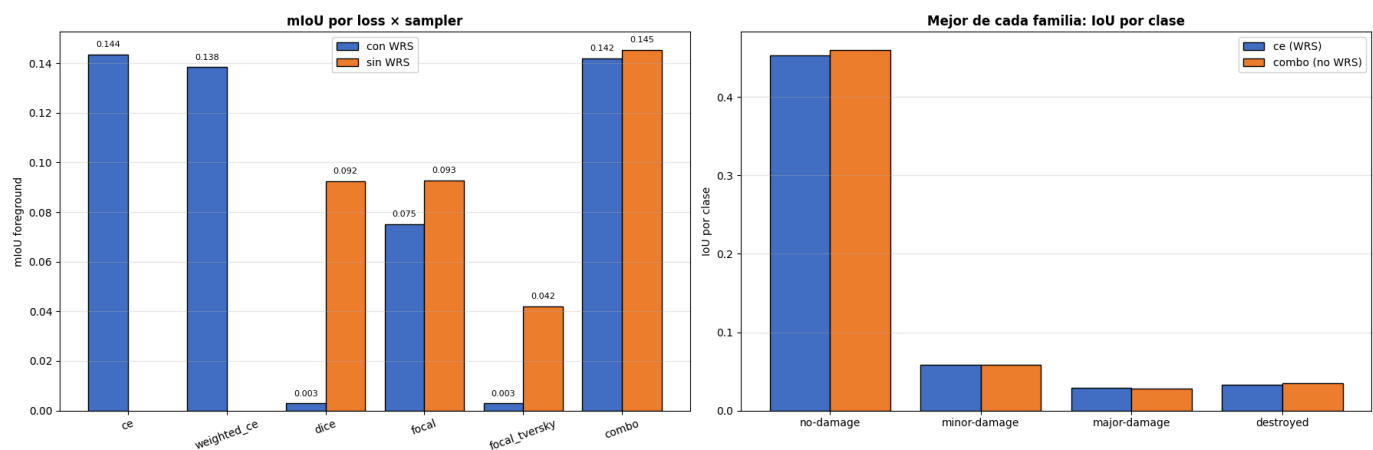
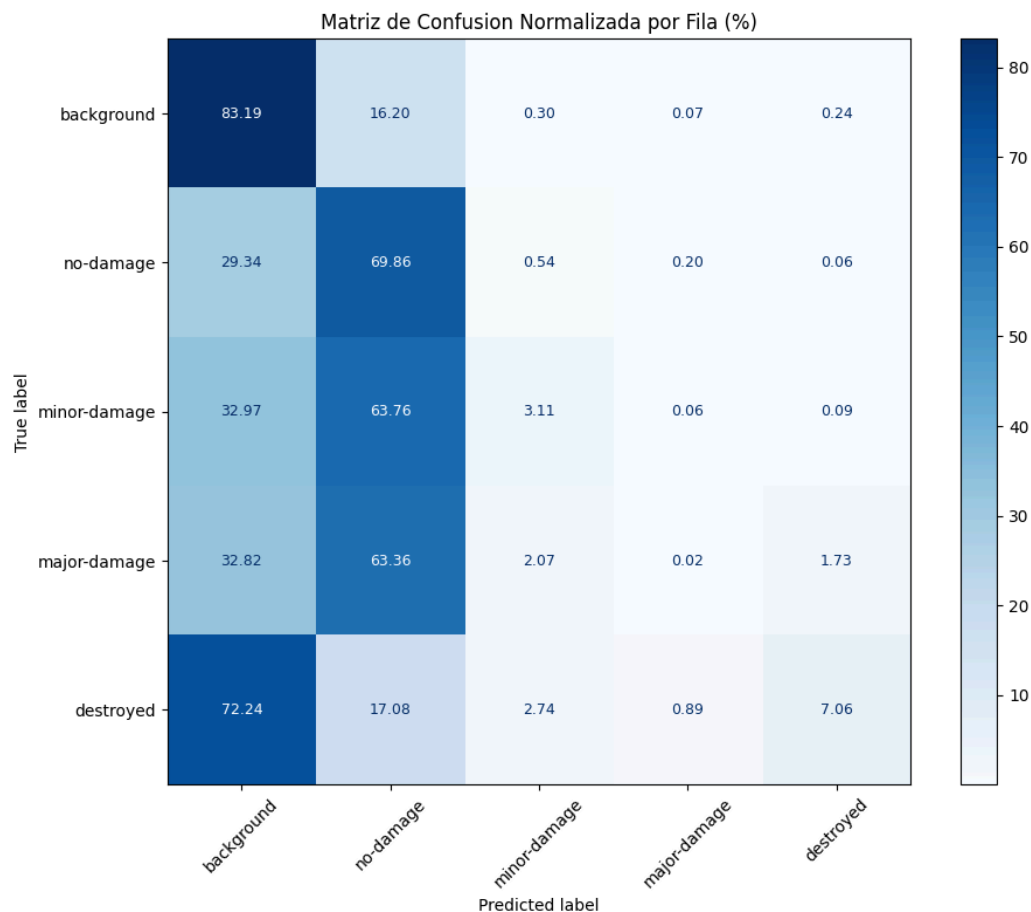


Figura 3:

Matriz de confusión normalizada por fila (%) del run final integrado (Combo + shuffle, 15 épocas) evaluado sobre test_with_labels (5 desastres no vistos). Las clases de daño se confunden sistemáticamente con background y no-damage.



REFERENCIAS

[1] R. Gupta et al., "xBD: A dataset for assessing building damage from satellite imagery", en Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, 2019.

PARTICIPACIONES

María Montanet: Diseño e implementación del módulo de funciones de pérdida (losses.py) con seis variantes y un selector unificado. Diseño y ejecución de una ablación 2D loss × sampler que identificó la interacción de doble balanceo entre WeightedRandomSampler y pérdidas regionales. Selección de Combo (CE+Dice) como pérdida final del proyecto y validación de la decisión sobre el conjunto de test con desastres no vistos.

Guillermo Sánchez: Evaluación de motores de optimización (validando la eficacia de AdamW para evitar el colapso inicial) y exploración de variantes topológicas sobre DeepLabV3 (clasificador auxiliar, ajuste de tasas ASPP) para mejorar la detección de daños severos.

Lara Gómez: Implementación de JointTransform (augment.py) para aumento de datos sincronizado imagen-máscara, ablación de cinco configuraciones de aumento y análisis del WeightedRandomSampler sobre clases minoritarias. Diseño del baseline compartido: re-split estratificado 85/15 y configuración del DataLoader de entrenamiento.